There are generally two approaches to random number generation (RNG): software-based and hardware-based. Software generation relies on deterministic algorithms (behavior is determined by input, passing through the same states) and is predictable. For this reason, software-based RNG is often called *pseudorandom*. In contrast, hardware-based RNG uses unpredictable (at least over the short term) physical processes such as isotope decay or radio static.

On Linux systems the special file `/dev/urandom` collects noise from hardware (via the device drivers) and is usually what is used by PHP's cryptographically secure random number functions. Let's take a look at `random_bytes()` and `random_int()`:

```
var_dump(random_int(0, 100));
var_dump(random_int(0, getrandmax()));
var_dump(random_bytes(1));
var_dump(random_bytes(8));

Output:
int(89)
int(1212946012)
string(1) "?"
string(8) "y./vGp?"
```

Naturally, the output varies on each execution (with true randomness!), but you get the idea. `random_int()` always requires a min and max argument, and `random_bytes()` always requires a length argument. Now you can enjoy cryptographically secure random numbers whenever you need them.

In addition to the built-in functions, PHP 8.2 also introduced an OOP interface for random numbers as part of its core. The new API is available in the built-in `\Random` namespace (for more info on namespaces and objects, see Chapter 6, "Object Fundamentals"). Here are a few examples; feel free to read up on how classes and objects work if you need to and return to this section:

```
$randomizer = new \Random\Randomizer();

var_dump($randomizer->getInt(0, 100));
var_dump($randomizer->getBytes(8));

Output:
int(30)
string(8) "???W?9"
```